РУКОВОДСТВО ДЛЯ РАЗРАБОТЧИКА · 29.10.2026

Приём DASH и подключение кошелька

DashPayBox + Dash Platform Extension

1. Архитектура интеграции

Для подключения оплаты нужны два компонента: микросервис DashPayBox на стороне магазина и расширение Dash Platform Extension у покупателя. Backend создаёт счёт, сайт вызывает requestSendPayment с адресом и суммой, кошелёк выполняет перевод. DashPayBox обнаруживает платёж и подтверждает заказ через webhook.

Компонент	Задача
Frontend магазина	Показывает счёт, QR-код, срок оплаты и статус заказа. Предлагает оплату через расширение при наличии requestSendPayment, а также QR-код и dash: URI.
Backend магазина	Определяет сумму заказа, создаёт счёт через API DashPayBox, проверяет webhook и выдаёт товар или доступ.
DashPayBox	Создаёт одноразовый адрес, фиксирует курс fiat → DASH, отслеживает платёж и переводит средства на адрес продавца.
Dash Platform Extension	Метод requestSendPayment показывает запрос оплаты, получает подтверждение покупателя и выполняет перевод. Приватные ключи остаются в кошельке.

Поток оплаты

Покупатель → checkout → backend магазина → DashPayBox → адрес и сумма → requestSendPayment или QR → подтверждение в кошельке → подписанный webhook → заказ оплачен.

Что подключает разработчик магазина

HTTP API DashPayBox на backend, обработчик подписанного webhook и одну кнопку на frontend с вызовом requestSendPayment. Устанавливать Dash SDK на сайт, создавать State Transition или реализовывать подпись транзакций не требуется: перевод выполняет расширение.

> **Статус DashPayBox:** версия 0.1.0, pre-1.0 / testnet / evaluation-grade. BLS-проверка InstantLock и ChainLock ещё не реализована; финальность определяется по данным DAPI-узла. Перед использованием для реальных платежей требуется устранить это ограничение. Для пилота используйте testnet и доверенные DAPI-узлы.

Инструкция составлена по локальному исходному коду DashPayBox 0.1.0 и Dash Platform Extension 1.5.0 с добавленным API оплаты. API оплаты проверен по worktree feat/requestSendPayment; возврат network и события — по feat/signerWithoutPolling. Для совместного использования нужен билд с обоими наборами изменений. Сквозная проверка с реальным платежом в рамках подготовки документа не проводилась.

2. Подключить приём оплаты

Шаг 1. Развернуть сервис

В каталоге DashPayBox соберите Docker-образ:

```
docker build -t dashpaybox:local .
```

Создайте файл `dashpaybox.env`, подставив адрес testnet и три независимых случайных секрета:

```
NETWORK=testnet
OWNER_STORAGE_ADDRESS=<адрес продавца в testnet>
AUTH_SECRET=<секрет API>
KEYS_ENCRYPTION_SECRET=<секрет шифрования ключей>
CALLBACK_URL=https://your-site.com/webhooks/dash
CALLBACK_SECRET=<секрет подписи webhook>
```

Например, каждый секрет можно получить командой `openssl rand -hex 32`. Храните файл вне репозитория с ограниченным доступом.

```
docker run -d --name dashpaybox --restart unless-stopped \
  --env-file dashpaybox.env \
  -p 127.0.0.1:8090:8090 \
  -v dashpaybox-data:/app/data dashpaybox:local
```

При таком запуске backend на том же хосте обращается к `http://127.0.0.1:8090`. Для backend в другом контейнере используйте общую Docker-сеть и адрес сервиса. Сохраняйте volume и секрет шифрования: они нужны для доступа к одноразовым ключам. DAPI-узлы обнаруживаются автоматически; `DAPI_SEEDS` позволяет задать доверенные узлы явно.

Шаг 2. Создать счёт из backend

```
const res = await fetch('http://127.0.0.1:8090/intents', {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json',
    'X-Dash-Auth': process.env.DASH_AUTH_SECRET,
  },
  body: JSON.stringify({
    order_id: 'order-123',
    amount_minor: 1500, // 15 USD в центах
    currency: 'USD',
    instant_send: true,
    min_confirmations: 1,
  }),
});
if (!res.ok) throw new Error(`DashPayBox: ${res.status}`);
const invoice = await res.json();
```

Сумму определяет backend по данным заказа. Сохраните связь `order_id ↔ intent_id`. Каждый POST создаёт новый счёт: повторный запрос с тем же `order_id` не является идемпотентным. Защитите создание счёта от повторных кликов и повторов на стороне магазина.

3. Подтвердить оплату

Шаг 3. Показать счёт покупателю

Ответ `POST /intents` содержит `intent_id`, `address`, `duffs`, `amount`, `rate`, `rate_source`, `uri` и `expires_at`.

- Покажите адрес и точную сумму `amount` в DASH. При наличии метода предложите кнопку «Оплатить через расширение» (раздел 4).
- Сформируйте QR-код из `uri` вида `dash:ADDRESS?amount=...` и ссылку «Открыть кошелёк» с этим URI.
- Покажите срок оплаты: по умолчанию 900 секунд. Покупатель может платить Dash-кошельком без установки расширения.
- Frontend запрашивает состояние заказа у вашего backend. Для сверки backend использует `GET /intents/:id` с `X-Dash-Auth`.

Шаг 4. Проверить webhook

На `CALLBACK_URL` приходит POST. Проверяйте подпись по исходным байтам тела до JSON-разбора. Пример функции для Node.js:

```
import { createHmac, timingSafeEqual } from 'node:crypto';

function verifyWebhook(rawBody, signature, secret) {
  if (typeof signature !== 'string' ||
      !/^[0-9a-f]{64}$/i.test(signature)) return false;
  const expected = createHmac('sha256', secret)
    .update(rawBody).digest();
  const received = Buffer.from(signature, 'hex');
  return expected.length === received.length &&
    timingSafeEqual(expected, received);
}
// rawBody: Buffer из HTTP-запроса
// signature: заголовок X-Dash-Signature
// secret: значение CALLBACK_SECRET
```

event	Обработка
confirmed	Заказ оплачен. Переплата также считается оплатой.
mismatch	Недоплата: автоматическую выдачу не выполнять.
expired	Срок счёта истёк без оплаты.
late	Поздняя оплата: автоматическая выдача не выполняется, решение о возврате принимается вручную.

После проверки подписи сопоставьте `intent_id` и `order_id` с сохранённым счётом. В одной транзакции БД сохраните событие с уникальным ключом `(intent_id, event)` и измените заказ. Повтор события — без повторной выдачи. Отвечайте 2xx после надёжного сохранения; сервис повторяет неуспешную доставку.

AUTH_SECRET и CALLBACK_SECRET не передаются в браузер. Средства консолидируются на OWNER_STORAGE_ADDRESS; у sweep есть комиссия и минимальный порог. Автоматические возвраты и автосписания по подписке не реализованы.

CHECKOUT · ОПЛАТА ЧЕРЕЗ РАСШИРЕНИЕ

4. Подключить кошелёк и оплатить

DashPayBox возвращает Core-адрес счёта. У покупателя должен быть seedphrase-кошелёк с достаточным Core-балансом. Сайт подключается к расширению, затем по отдельному нажатию запрашивает оплату. Каждая оплата подтверждается пользователем в кошельке.

```
const ext = window.dashPlatformExtension;
if (typeof ext?.requestSendPayment !== 'function') {
  // Покажите QR-код и dash: URI — метод отсутствует в билде.
  throw new Error('Обновите расширение или оплатите по QR');
}

// Кнопка «Подключить кошелёк».
const connected = await ext.signer.connect();
// Для билда с возвратом network:
const expectedNetwork = 'testnet'; // сеть магазина / DashPayBox
let walletNetwork = connected.network;
const canPay = () => walletNetwork === expectedNetwork;

// При несовпадении отключите оплату и покажите подсказку:
// «Выберите Testnet в Dash Platform Extension».
// При отсутствии network обновите билд расширения.

// Кнопка «Оплатить», после подключения и проверки сети.
async function pay(invoice) {
  if (!canPay()) throw new Error('Выберите нужную сеть в кошельке');
  if (Date.now() >= Date.parse(invoice.expires_at)) {
    throw new Error('Срок счёта истёк');
  }
  const result = await ext.requestSendPayment({
    address: invoice.address,
    amountDuffs: String(invoice.duffs),
  });
  // Ожидайте confirmed от backend, даже при delivery === 'sent'.
  return result;
}
```

Сеть сайт не передаёт и не переключает: при несовпадении просит пользователя выбрать её в кошельке. В билде с событиями подпишитесь на `ext.on('networkChanged', ({ network }) => { walletNetwork = network; /* обновить UI */ })`. При смене контекста повторно подключите сайт при необходимости. При уходе со страницы удалите обработчик через `off`.

Параметры и результат

`amountDuffs` — положительная целочисленная строка: 1 DASH = 100 000 000 duffs. Это сумма получателю, комиссии оплачиваются сверху. Передавайте `String(invoice.duffs)` без пересчёта из отображаемого DASH.

```
{
  id: '...', address: '...', amountDuffs: '10000000',
  delivery: 'sent', // либо 'queued' для отложенного withdrawal
  transactions: [{ type: 'core', txid: '...' }]
  // Platform: { type: 'platform', stateTransitionHash: '...' }
}
```

Сейчас в проверенном API доступен Core → Core. Распознавание Platform и Orchard уже есть, но платежи по этим маршрутам пока отклоняются. delivery: sent означает отправку, queued — принятие отложенного вывода; подтверждение заказа всегда приходит от DashPayBox.

Блокируйте повторное нажатие. Каждый вызов создаёт отдельный платёж. Отказ, закрытие окна до подтверждения, истечение запроса (10 минут) и смена сети/кошелька до подтверждения отклоняют Promise. После подтверждения операция продолжает выполняться при закрытии окна: при потере ответа сначала проверьте заказ, не платите автоматически повторно.

ИНТЕГРАЦИЯ · ПОЛНЫЙ СЦЕНАРИЙ

5. Связать компоненты

Backend магазина

1. По запросу авторизованного покупателя найдите заказ и определите сумму из своей базы.
2. Создайте счёт через POST /intents в DashPayBox. Сохраните intent_id вместе с заказом и верните frontend адрес, duffs, uri и expires_at.
3. На webhook проверьте HMAC и принадлежность счёта заказу. По confirmed атомарно отметьте заказ оплаченным и сохраните событие для защиты от повторной выдачи.
4. Предоставьте frontend свой endpoint статуса заказа. Не передавайте браузеру секрет доступа к DashPayBox.

Frontend магазина

```
// Пример endpoints магазина — их реализует ваш backend.
const invoiceResponse = await fetch(
  '/api/orders/order-123/dash-invoice', { method: 'POST' },
);
if (!invoiceResponse.ok) throw new Error('Не удалось создать счёт');
const invoice = await invoiceResponse.json();

// Сохраните invoice и покажите сумму, срок оплаты и кнопку.
// Сначала подключите кошелёк и проверьте сеть (раздел 4).
// По нажатию кнопки «Оплатить»:
const extension = window.dashPlatformExtension;
if (typeof extension?.requestSendPayment === 'function') {
  const result = await extension.requestSendPayment({
    address: invoice.address,
    amountDuffs: String(invoice.duffs),
  });
  // Покажите «Ожидаем подтверждения оплаты».
  // result не используется как доказательство оплаты заказа.
} else {
  // Покажите QR-код из invoice.uri и адрес для копирования.
}

// Запрашивайте статус у backend магазина:
// GET /api/orders/order-123/status
// Оплачено — только когда backend обработал confirmed.
```

Пример иллюстрирует поток; обработку ошибок, блокировку повторного нажатия и проверку expires_at добавьте в UI, как описано в разделе 4. Endpoints магазина должны проверять права покупателя на заказ.

Что делает покупатель

Устанавливает расширение, выбирает нужную сеть и пополняет кошелёк. На checkout нажимает «Оплатить через Dash Platform Extension», проверяет сумму и комиссию в кошельке и подтверждает. Создание, подпись и отправку транзакции выполняет расширение. В текущем API оплаты используется Core-баланс.

> **Граница интеграции:** магазин выставляет счёт и ждёт подтверждение, расширение выполняет перевод, DashPayBox отслеживает поступление. Создавать транзакции через SDK, управлять nonce, ключами или State Transition разработчику магазина не нужно.

Исходники: github.com/lexxxell/dashpaybox · github.com/pshenmic/dash-platform-extension
Примеры backend и webhook: dashpaybox/examples/. Проверьте состав билда: requestSendPayment, возврат network из connect и события networkChanged разрабатывались в отдельных ветках. SDK магазину не требуется.